

The ethics of generative AI in software engineering

Edgar Jose Donoso Mansilla

e.donosomansilla@digipen.edu

DigiPen Institute of Technology

USA

ACM Reference Format:

Edgar Jose Donoso Mansilla. 2026. The ethics of generative AI in software engineering. In . ACM, New York, NY, USA, 6 pages. <https://doi.org/10.1145/nnnnnnn.nnnnnnn>

1 Introduction

Software engineering is a field that has developed most of today's infrastructure in some way or another, be it via the implementation of a medical device to the websites that allow us to connect the world. However, these projects need to manage several stakeholders, engineers and budgets to make things come to reality. With the improvements in generative AI (GenAI for short), there has been a shift towards trying to incorporate this new tool into the software development pipeline. These attempts are not small companies but instead industry leaders such as Microsoft. [21]

Given the workings of generative AI, there are several key concerns: The displacement of people for compute centers, the reliability of the software

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

Conference'17, Washington, DC, USA

© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM ISBN 978-x-xxxx-xxxx-x/YYYY/MM

<https://doi.org/10.1145/nnnnnnn.nnnnnnn>

generated and the dilution of understanding the software developed. This essay will try to answer the question:

“Is it ethical for software engineers to use modern generative AI applications to generate code that is used, in whole or in part, for commercial applications?”

2 Core Issues

As discussed in the introduction, there are several concerns with the use of generative AI in software development:

2.1 Software reliability

One of the major issues that stems from using GenAI is the fact that GenAI doesn't *reason* like humans do given its answer is generated via approximating using other good answers in the source. Programming is a task that requires discrete reasoning as the steps will directly relate to the next one in the solution [6]. This causes several issues when it comes down to the reliability of the software being written with the help of GenAI [13]:

- Mistakes might be missed when generating large amounts of code.
- Engineers can't guarantee where the design patterns that were used to generate the system.
- Maintaining the systems will become more difficult over time as they were mainly written by GenAI as opposed to a human who had a stream of logic that was followed when designing the system.

These issues are more prominent when the software with generated code has critical importance such as a pacemaker, airplane cruising modes, etc. This issue compounds with GenAI's tendency to "hallucinate" an answer as its model might have biases and over fitting which make the math converge to a wrong answer [12].

2.2 Dilution of knowledge

Another major issue with GenAI for software engineering is the fact that they reduce the spaces where less experienced engineers build the knowledge to move forward. As stated before, GenAI doesn't reason and this is a similar issue with less experienced engineers, however, the key difference is that human engineers can improve with time and build up the discrete reasoning which allows them to construct correct programs that they haven't seen before. [16]

2.3 Displacement of people

This is a more general issue with the usage of GenAI but applies heavily to expensive tasks like generating the code for a complex program. The infrastructure that GenAI needs is costly in several resources [15, 22]:

- About 22% power of all households in the US.
- "Chan School of Public Health found that the carbon intensity of electricity used by data centers was 48% higher than the US average."
- "Large data centers can consume up to 5 million gallons per day, equivalent to the water use of a town populated by 10,000 to 50,000 people."

Not only are data centers resource expensive, they also need a lot of space. This has caused local communities to be affected by the pollution and price increase of utilities. Not only that but there are cases where people lose the green areas of their community to a company who was able to buy out the land [5].

3 Stakeholders

With the core issues of GenAI out of the way, these are a possible classification of the different stakeholders in the situation (in order of least to most directly benefited):

- (1) General population in developing countries
- (2) General population in developed countries
- (3) Software engineers
- (4) Companies that use generative AI
- (5) Companies that make generative AI tools

4 Current state of generative AI

With understanding of the costs of this technology it is also important to acknowledge its usage. By understanding the applications of the technology, it will be possible to weigh out the benefits and disadvantages of spending resources on this technology.

4.1 Vibe Coding

This no-code development experience differs from the professional world of software engineering where the tools are understood. This really doesn't have a major impact in the infrastructure of larger software but it does use all the same resources that other projects would use to improve their products [9].

4.2 Usage in research

There's been a shift in academia to use more GenAI to speed up different elements of the research pipeline [1]. A key project that will be mentioned in this essay is the usage to create the implementation for the ZOZO's (a Japanese clothing company) cloth physics solver [2, 3].

4.3 Usage in open source

In open source projects there is a current issue where the lead contributors are being swarmed by pull requests with low quality AI generated code [14]. This puts a strain in projects' ability to

survive as the leads are often people who volunteer and don't have the time to go through all the low quality code to pick out and accept contributions to the project.

4.4 Usage in private companies

Across different companies, there are several philosophies, some are encouraging their developers to use it and needing to strengthen their code reviews [7] and other companies avoid it as they feel it damages the quality of their products [11].

5 Technical Background

5.1 Machine learning

Machine learning is a subset of artificial intelligence that uses statistical inference to adapt responses based on previous and new knowledge. This is a very powerful type of program as it provides adaptability at no code architecture change, however, these systems improve drastically the more data it has available and analyzed [8].

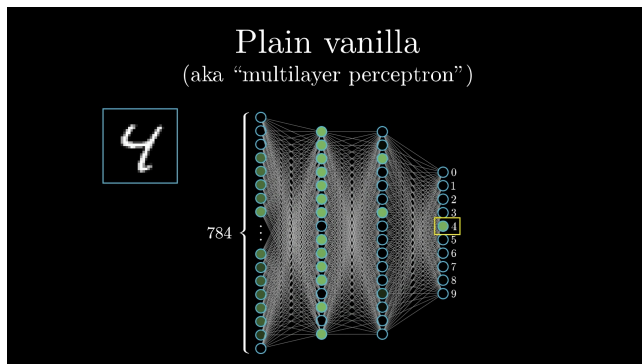


Figure 1: Neural network made to recognize numbers 0 to 9 on a 28 by 28 grid [17].

Models work via the implementation of neural networks. A neural network is composed of *neurons*, which are simply just a number that represents the presence of a component in the data (the circles in the image 1). A vector of neurons represents a layer, this is used to represent different

components of the data. Neural networks are normally composed of several layers each neuron connected to another via weights (the lines in image 1). This allows the model to encode the activation of a neuron based on a linear combination of the weights and neurons of the previous layer. In the image 1 the activation is represented by how green each circle is. Each layer encodes different sets of properties about the data, in the case of figure 1 the first layer represents how bright each pixel is while the next one represents relationships in the distributions. The first layer is one of the few layers that can be explicitly linked to the data as the activation is entirely determined by the model maker. This is because these linear combinations can be fine tuned with a bias term to ensure that the model produces better results [17].

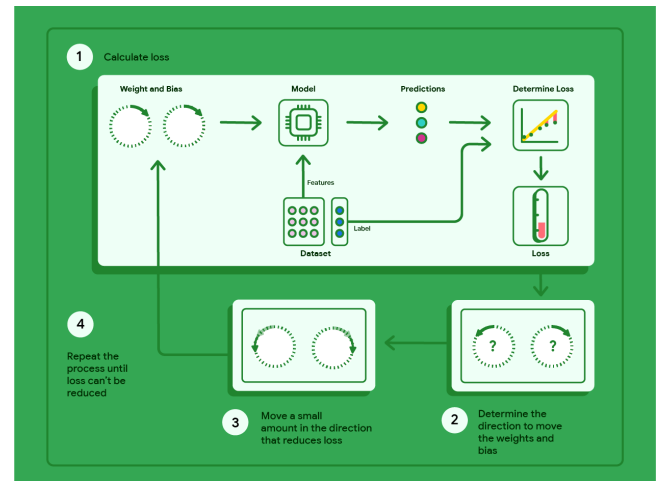


Figure 2: The model training pipeline [10]

The training process for a model involves utilizing data that is labelled to be the right response and iteratively improving all of the model's weights and biases (denoted by $\vec{w}$). This process of improvement utilizes cost function and gradient descent. The cost function (ϵ) describes the error of the current model based on the given parameters, this error is normally averaged to improve across all input types instead of only one [10]. Gradient descent is an iterative optimization method defined as such:

$$\vec{w}' = \vec{w} + \gamma \begin{bmatrix} \frac{\partial \epsilon}{\partial w_1} \\ \vdots \\ \frac{\partial \epsilon}{\partial w_n} \end{bmatrix}$$

This method is used for steps 2 and 3 in figure 2 as it will calculate the direction to move the weights and biases and move there. This process is repeated until the change in error between iterations is less than desired [10]. While this only adjusts the weights, there still needs to be a process of back-propagation. This involved finding which parameters affect the output the most and then adjusting the values to cover the edge cases left after the gradient descent is done ¹ [18].

5.2 Large Language Models

Large Language Models (LLMs) are models that are trained on natural and computer language in order to be able to generate an output based on verbal (text and/or audio) input. These models use the relationships encoded with machine learning to output next most likely word [19].

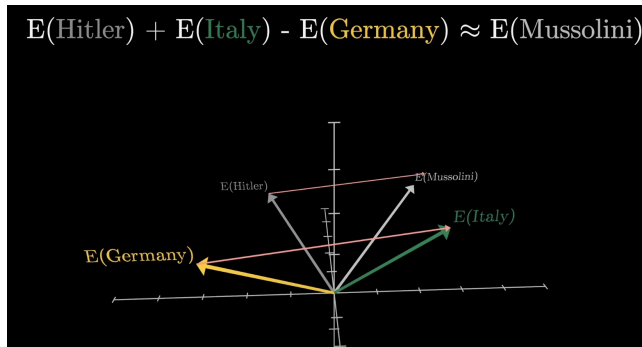


Figure 3: Visuals of how the embedding matrices transform different words into different "word spaces"[19]

When the user submits a prompt, the model will turn the prompt into tokens. Tokens are small chunks of the prompt that can be turned into vector values (each word being like a pixel in figure 1).

¹The math of this algorithm is more complex therefore it will not be explored in depth for this essay

This is done with an embedding matrix, which describes the conversions between a token and their respective vector value. This vector value, like discussed in the machine learning section, represents a set of relationships be it about the word itself or the semantic context of the word [19].

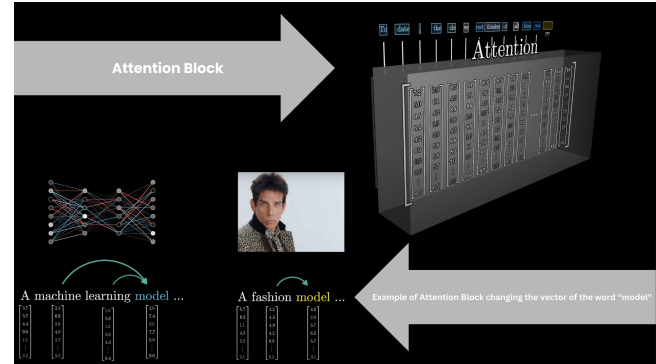


Figure 4: Visuals of how the attention block uses and represents context numerically [19]

After that, each vector will affect each other to ensure that the context is taken into account, this is known as the attention block (see figure 4) [19].

After finalizing the contextualized vectors, the model will then pass each vector separately through the layers of the neural network. This will result with a final vector like the one in figure 1, but the vector will represent a scalar of how relevant the word is.

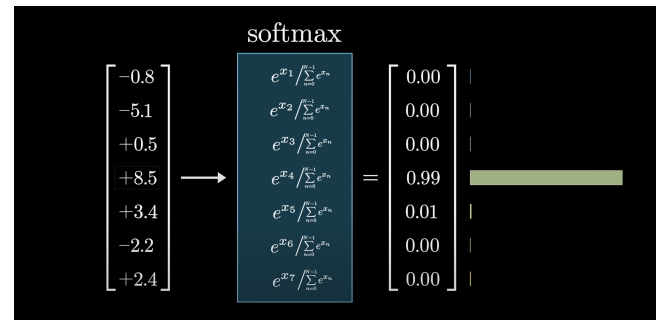


Figure 5: How the scalar values are turned into a probability distribution [19]

As the scalars are not a probability, the procedure in figure 5 is performed. This then allows the model to pick either the most likely word with a bit of

added randomization to mimic natural language [19].

This process is conducted iteratively until the response is fully formed. The nice thing about this model is how it can turn the set of input tokens into a set of different parameters used to construct an output. While this example focused on generating text, this extends to code with the main difference being that the output vector is not composed of words or punctuation but symbols and language tokens (such as brackets or keywords) [20].

5.3 Architecture of GenAI coding tools

When utilizing generative AI in the context of a software project, there are additional considerations that must be met to meet the desired output. To better understand this fine tuning, this essay will focus on Claude Code's approach as a case study.

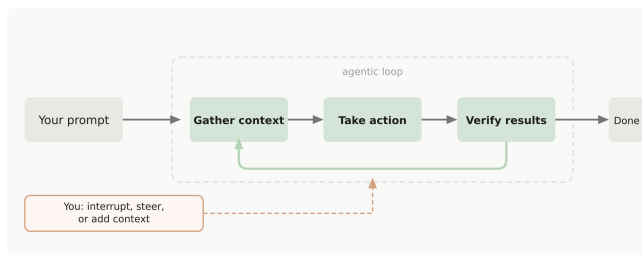


Figure 6: Claude Code's prompt processing architecture [4]

When a developer prompts a Claude Code, the tool will start by gathering context of the problem. This involves analyzing the codebase for relevant symbols and classes that are needed to complete the required prompt. Then the tool will execute actions such as generating text or code. Lastly, because Claude has permissions to interact with the terminal, it will try to verify the results by either compiling the code, running the code and verifying any output produced by the program [4].

While not all developers directly allow GenAI tools to access their codebase like in figure 6, this is the most effective at producing responses as it

is able to gather more information and test the generated content.

6 Ethical Analysis

6.1 Kantianism

Under Kantianism, the biggest ethical concerns are:

- Using code for training without the owner's consent (GitHub Copilot)
- Displacing people to fit more compute centers
- Using more water to keep compute centers cool

6.2 Act Utilitarianism

In act utilitarianism the biggest contrast is whether all the negative effects on other humans helps maximize the happiness in other humans. This can be evaluated through the development of new technologies and research. By evaluating different sources it is possible to evaluate such cases.

6.3 Virtue Ethics

Under virtue ethics, the focus is the virtues of the stakeholders and how the decisions they make with this technology and how virtues either align or misalign with them.

6.4 Social Contract Theory

This section focuses on what our society expects out of software and how the usage of machine learning can breach the agreements and expectations that we have about the software that we use and how it is made.

7 Conclusion

8 Beyond the surface

References

- [1] Jens Peter Andersen, Lise Degn, Rachel Fishberg, Ebbe K. Graversen, Evanthia Kalpazidou Schmidt Serge P.J.M. Horbach b, Jesper W. Schneider, and Mads P. Sørensen. [n. d.]. *Generative Artificial Intelligence (GenAI) in the research process – A survey of researchers' practices and perceptions*. <https://cloud.google.com/discover/what-is-vibe-coding>
- [2] Ryoichi Ando. [n. d.]. *ZOZO's Contact Solver*. <https://github.com/st-tech/ppf-contact-solver>
- [3] Ryoichi Ando. 2024. *A Cubic Barrier with Elasticity-Inclusive Dynamic Stiffness*. doi:doi.org/10.1145/3687908
- [4] Anthropic. [n. d.]. *How Claude works*. <https://code.claude.com/docs/en/how-claude-code-works>
- [5] Pablo Jiménez Arandia. [n. d.]. *How We Investigated the Backyard of AI in Spain, Chile, and Mexico*. <https://pulitzercenter.org/resource/how-we-investigated-backyard-ai-spain-chile-and-mexico>
- [6] R&A IT Strategy & Architecture. [n. d.]. *Generative AI 'reasoning models' don't reason, even if it seems they do*. <https://ea.rna.nl/2025/02/28/generative-ai-reasoning-models-dont-reason-even-if-it-seems-they-do/>
- [7] AWS. [n. d.]. *AWS Responsible AI Policy*. <https://aws.amazon.com/ai/responsible-ai/policy/>
- [8] Chrystal R. China. N/A. *Five machine learning types to know*. <https://www.ibm.com/think/topics/machine-learning-types>
- [9] Google Cloud. [n. d.]. *What is Vibe Coding?* <https://cloud.google.com/discover/what-is-vibe-coding>
- [10] Google Developers. [n. d.]. *Linear regression: Gradient descent*. <https://developers.google.com/machine-learning/crash-course/linear-regression/gradient-descent>
- [11] Peter Glagowski. [n. d.]. *Nintendo clarifies its stance on generative AI, says it has not had any contact with Japanese government over its usage*. <https://nintendowire.com/news/2025/10/06/nintendo-clarifies-its-stance-on-generative-ai-says-it-has-not-had-any-contact-with-japanese-government-over-its-usage/>
- [12] IBM. [n. d.]. *What are hallucinations?* <https://www.ibm.com/think/topics/ai-hallucinations>
- [13] Baskhad Idrisov and Tim Schlippe. [n. d.]. *Program Code Generation with Generative AIs*. doi:10.3390/a17020062
- [14] Arjun Iyer. [n. d.]. *Open source maintainers are drowning in AI-generated pull requests. Enterprise teams are next*. <https://thenewstack.io/ai-generated-code-crisis/>
- [15] James O'Donnellarchive and Casey Crownhart. [n. d.]. *We did the math on AI's energy footprint. Here's the story you haven't heard*. <https://www.technologyreview.com/2025/05/20/1116327/ai-energy-usage-climate-footprint-big-tech/>
- [16] Phoebe Sajor. [n. d.]. *AI vs Gen Z: How AI has changed the career pathway for junior developers*. <https://stackoverflow.blog/2025/12/26/ai-vs-gen-z/>
- [17] Grant Sanderson and Josh Pullen. 2017. *But what is a Neural Network?* <https://www.3blue1brown.com/lessons/neural-networks>
- [18] Grant Sanderson and Josh Pullen. 2017. *What is backpropagation really doing?* <https://www.3blue1brown.com/lessons/backpropagation>
- [19] Grant Sanderson and Josh Pullen. 2024. *Transformers, the tech behing LLMs*. <https://www.3blue1brown.com/lessons/gpt>
- [20] Ian Smalley. N/A. *What is AI code-generation software?* <https://www.ibm.com/think/topics/ai-code-generation>
- [21] Jaime Teevan, Sonia Jaffe, Rebecca Janssen, Nancy Baym, Siân Lindley, Bahar Sarrafzadeh, Brent Hecht, Jenna Butler, Jake Hofman, and Sean Rintel. [n. d.]. *New Future of Work: AI is driving rapid change, uneven benefits*. <https://www.microsoft.com/en-us/research/blog/new-future-of-work-ai-is-driving-rapid-change-uneven-benefits/>
- [22] Miguel Yañez-Barnuevo. [n. d.]. *Data Centers and Water Consumption*. <https://www.eesi.org/articles/view/data-centers-and-water-consumption>